

1. Experiments & Justification

Exp 1 — Horizontal Auto-Scaling: Locust simulates flash-sale bursts against ECS deployments (1/2/4/8 tasks). Measures whether auto-scaling reacts fast enough before users see errors. Metrics: throughput, latency (p50/p95/p99), error rate, scaling reaction time.

Exp 2 — Locking Strategies: Compare pessimistic (row-level lock), optimistic (version-based conditional update), and Redis distributed lock under identical flash-sale load. Under low contention any strategy works — this exposes real costs (retry storms, connection starvation) at scale. Metrics: correctness (zero overselling), throughput, retry rate, p99 latency.

Exp 3 — Caching Layer: Redis cache on Event Service reads, toggled via Terraform. Flash-sale reads vastly outnumber writes and compete for DB connections the Reservation Service needs — this quantifies the actual offload. Metrics: DB load reduction, read latency, cache hit rate, staleness.

Exp 4 — Consistency Under Failure: Kill ECS tasks and inject partitions during peak load, then verify DB state. Experiments 1-3 assume healthy infra — this validates that our CP model actually holds during failures. Validation: zero overselling, zero corruption, recovery time.

2. Team Value Beyond AI

We use AI (Claude, Copilot) for boilerplate and debugging. The core work is ours:

- **Architecture tradeoffs** — choosing CP over AP, selecting locking strategies, cache invalidation design require domain-specific judgment.
- **Experiment design** — structuring tests to isolate variables and produce actionable conclusions.
- **Infra debugging** — ECS misconfigs, Terraform state drift, security groups are context-dependent problems.
- **Result interpretation** — deciding if a p99 increase is acceptable given a throughput gain is engineering judgment, not generation.
- **Load modeling** — realistic flash-sale patterns (spike shape, request mix) require domain understanding.

3. Timeline

- **Wk 1-2:** Core Go services + Terraform ECS/ALB/RDS deployment
- **Wk 3:** Exp 1 — Scaling tests
- **Wk 4:** Exp 2 — Locking comparison

- **Wk 5:** Exp 3 — Cache experiments
- **Wk 6:** Exp 4 — Chaos testing
- **Wk 7:** Analysis, report, presentation